Chapter 5: Writing Custom Predicates

Rod Trent

Chapter 5: Writing Custom Predicates

Download this chapter as a PDF for offline reading:
Chapter-05-Writing-Custom-Predicates.pdf

Predicates are the reusable building blocks of QL. This chapter shows how to write them well, how to organize them, and how to turn one-off query logic into predicates you can share across queries.

Predicates as Reusable Units

A predicate captures a logical relation you can name and reuse. Instead of repeating the same `where` clause in every query, you extract it into a predicate.

Non-member predicates

```
predicate isSmall(int i) {
  i in [1..10]
}

int square(int i) {
  result = i * i and isSmall(i)
}
```

Member predicates

Member predicates belong to a class and can use `this`. They are especially convenient for chaining:

```
class SmallInt extends int {
  SmallInt() { this in [1..10] }

  int square() { result = this * this }
}
```

Characteristic predicates

The characteristic predicate of a class (the one with the same name as the class) defines which values belong to the class:

```
class PositiveInt extends int {
  PositiveInt() { this > 0 }
}
```

Why extract predicates?

- **Reuse** — write the logic once, call it from many queries.
- **Readability** — a well-named predicate documents intent.
- **Testing** — you can unit-test a predicate in isolation.
- **Performance** — the compiler can cache and optimize shared predicates.

Examples and Organization

Naming conventions

Follow the official QL style guide:

- Predicates that return a result: `get`… (or `getA`… / `getAn`… when they can return multiple values).
- Predicates that only constrain values: `is`… or `has`….
- Use camelCase for predicate and variable names.
- Use PascalCase for classes and modules.
- Prefer American English and avoid underscores.

Where to put predicates

- **Inside a query file** — for logic used by only that query.
- **In a `.qll` library file** — for predicates you want to share across multiple queries.
- **In a module** — to group related predicates and avoid name clashes.

Example library file (`examples/chapter05/SmallInt.qll`):

```
class SmallInt extends int {
  SmallInt() { this in [1..20] }

  int square() { result = this * this }

  predicate isEven() { this % 2 = 0 }
}
```

A query that imports it (`examples/chapter05/even-squares.ql`):

```
/**
 * @name Even squares of small integers
 * @kind problem
 * @id mustlearn/even-squares
 */
import SmallInt

from SmallInt i
where i.isEven()
select i, i.square(), "Even square"
```

Practical tips

- Start with the logic inline in a query, then extract a predicate once you need it a second time.
- Keep predicates small and focused; compose them rather than writing large monolithic predicates.
- Document non-obvious predicates with a QLDoc comment (`/** ... */`).
- Prefer member predicates when the logic is naturally about a particular type.

Next Steps

Custom predicates let you build a personal library of reusable analysis building blocks. In the next chapter we put those pieces together to write complete, metadata-rich queries that produce alerts.

All examples referenced above live in the companion repository: github.com/rod-trent/MustLearnCodeQL under `examples/chapter05/`.

Must Learn CodeQL — Rod Trent
Companion repository: github.com/rod-trent/MustLearnCodeQL